

Makemore

Part 2 - The Multi-Layer Perceptron

Following Bengio et al., 2003

A comprehensive, cell-by-cell study handbook for the second video in Andrej Karpathy's *makemore* series — “Building makemore Part 2: MLP.” This expanded edition explains the **what**, the **why**, and the **how** behind every concept, code cell, and design choice in the notebook.

Compiled by: **Shreyas Pradeepkumar Khandale** · GitHub: **sherurox**

Companion to: `makemore2.ipynb`

Edition: Expanded - May 2026

Where this handbook sits in the series

Part 1 built a bigram model two ways — by counting and by training a tiny neural network — and showed they converge to the same matrix. The final loss was about **2.45**, and the names sampled from it (*mor*, *axx*, *minaymoryles*) sounded vaguely like names but were obviously broken. The problem was structural: with only one character of context, the model cannot see far enough to write good names.

Part 2 fixes that. We follow Bengio et al.'s 2003 paper *A Neural Probabilistic Language Model* — the paper that introduced learned word embeddings — and build a multi-layer perceptron that takes **three previous characters** as context. We end up with a loss of **2.20**, a real improvement, and we learn most of the deep-learning workflow vocabulary along the way: embeddings, hidden layers, `F.cross_entropy`, minibatches, learning rate finders, learning rate decay, and the train/dev/test split.

What is actually new compared to Part 1

- **Embeddings.** Instead of one-hot vectors, each character now lives in a learned low-dimensional space (a vector of 2, then 10 floats). This is the same primitive that scales all the way up to GPT.
- **Context windows.** The input is a sliding window of `block_size` previous characters, not just one. This is the model's “memory.”
- **A real hidden layer.** We add a `tanh` non-linearity between two linear layers. Now the network can model interactions between context positions.

- `F.cross_entropy`. Replaces our hand-rolled softmax + NLL with a single fused, numerically-stable call.
- **Minibatches**. We stop computing the full-batch gradient on 228k examples per step and use 32-example minibatches. Less accurate per step, but vastly more steps per second.
- **Learning rate search**. A clever trick using `torch.linspace` to discover a good learning rate empirically rather than guessing.
- **Train/dev/test split**. Without this, you cannot tell underfitting from overfitting, and tuning hyperparameters becomes superstition.

Preface: How to use this handbook

This handbook is organized as a guided walk through the `makemore2.ipynb` notebook, with each chapter mapping to a coherent group of cells in the notebook. It is designed to be read in either of two ways:

Reading mode 1: Linear study

Read the handbook end-to-end while keeping the notebook open in a separate window. Each chapter introduces the concept, presents the relevant code cell exactly as it appears in the notebook, and then explains what the code is doing, why it is structured that way, and how it fits into the larger training pipeline. This is the recommended path the first time you study the lecture, as it provides the full conceptual scaffolding required to understand each design decision.

Reading mode 2: Reference lookup

Once you have completed a first pass, the handbook also functions as a reference manual. The Table of Contents lists every concept with its chapter number, and the Glossary at the end provides one-line definitions for every technical term used in the lecture. The Diagnostic Flowchart in Chapter 29 is intentionally compact — refer to it when you encounter unexpected training behaviour in your own projects.

Conventions used in this handbook

The handbook uses a small set of visual conventions consistently throughout the document. Understanding these conventions before you begin will help you absorb information faster.

- **Code blocks** appear in a fixed-width font on a warm beige background. Code is reproduced verbatim from the notebook so you can match each block to the cell that produced it.
- **Cell labels** precede every code block in small blue type (for example, “*Cell — Initialize the embedding matrix*”). These labels indicate which logical cell of the notebook the code belongs to.
- **Callout boxes** (yellow background, rust-coloured title) highlight Karpathy's spoken framing, important warnings, or moments of insight. They are not optional reading.
- **Info boxes** (light blue background) contain conceptual definitions and theoretical background. They are useful for first-time learners and can be skimmed by readers already familiar with the material.
- **Part dividers** separate the five major sections of the handbook with a full-width banner.
- **Diagnostic cards** (Chapter 24) present each underfitting/overfitting scenario in its own clearly bounded card with three labeled rows: symptom, diagnosis, and action.

Prerequisites

This handbook assumes you have completed the Part 1 handbook (or are comfortable with the bigram model, negative log-likelihood, manual softmax, and basic PyTorch tensors). It also assumes a working knowledge of Python, including indexing, list comprehensions, and the standard library random module. Mathematical prerequisites are minimal: you should be comfortable with matrix multiplication, basic calculus (specifically the chain rule), and the idea that we are minimizing a loss function by computing its gradient with respect to parameters.

How to extract maximum value from this material

The single most effective learning strategy with Karpathy's lectures is to type out each cell of the notebook yourself rather than copy-pasting from the video. This handbook supports that strategy by explaining every cell in enough depth that you can rebuild the notebook from scratch using only this document as a reference. Once you have done that, the cleanest validation that you understood the material is to vary one hyperparameter (block size, hidden dimension, learning rate schedule) and explain to yourself, before running it, whether you expect train loss and dev loss to go up or down.

Contents

Front matter

Preface: How to use this handbook

PART I — Foundations

1. Why we have to move beyond the bigram
2. The Bengio 2003 paper at a glance

PART II — Building the architecture

3. Setup: imports, data, stoi and itos
4. Building the dataset with context windows
5. The embedding matrix C
6. Embedding lookup: indexing equals one-hot matmul
7. Embedding the entire dataset in one operation
8. The hidden layer: W1, b1
9. The flattening problem — (32, 3, 2) is not (32, 6)
10. Three ways to flatten: cat, unbind+cat, .view
11. Why .view is the winner
12. The tanh non-linearity
13. The output layer and logits

PART III — Training the model

14. Manual softmax and NLL — the slow way
15. Making it respectable — parameter lists and generators
16. F.cross_entropy — efficiency and numerical stability
17. The first training loop — overfitting on purpose
18. Scaling up: full dataset of 228k bigrams
19. Minibatches with torch.randint
20. The learning rate finder
21. Reading the LR-vs-loss plot — choosing 0.1
22. Learning rate decay

PART IV — Diagnostics and scaling

23. The train / dev / test split — what each one is for
24. Underfitting: when train loss equals dev loss
25. First scale-up: 100 to 300 hidden neurons
26. Visualizing the 2-D embedding space

- 27. Second scale-up: 2-D to 10-D embeddings
- 28. Beating the bigram baseline — first sign of overfitting

PART V — Conclusion and reference

- 29. Summary and key takeaways
- 30. Glossary

PART I

Foundations

Why we are moving from counting to neural networks — the theoretical motivation, the curse of dimensionality, and the architecture we will build.

1. Why we have to move beyond the bigram

Chapter overview

This chapter establishes the theoretical motivation for abandoning count-based language modelling and adopting a parametric, neural approach. Before we touch any PyTorch code, it is essential to understand *why* the bigram baseline — despite functioning as a working model in Part 1 — represents an architectural dead end. The argument is not that the bigram is buggy; it is that the model class itself cannot improve beyond a certain ceiling. This chapter walks through that ceiling and motivates the model we will spend the rest of the handbook building.

Recap of Part 1 and its limitation

Karpathy opens the video with a direct admission: the bigram model is bad. Not because the implementation was wrong, but because the architecture itself cannot see far enough to write believable names. *mor*, *axx*, and *minaymoryles* are the best it could do.

What is a bigram model?

A bigram model predicts the next character using exactly one previous character as context. It can be implemented as a 27×27 lookup table of counts (Part 1, first half) or as a 27×27 weight matrix trained by gradient descent (Part 1, second half). Both implementations are mathematically equivalent and produce the same final loss of approximately 2.45 nats per character. The model has no memory beyond the immediately preceding character.

The natural fix — use more context — runs into a wall when you try to do it by counting:

Context size	Combinations to count	Status
1 char (bigram)	27	fine - the Part 1 model
2 chars (trigram)	$27^2 = 729$	still fine
3 chars	$27^3 = 19,683$	barely tractable
5 chars	$27^5 \approx 14$ million	most combinations never seen
10 chars	$27^{10} \approx 2 \times 10^1$	impossible

The counting table grows *exponentially* with context length. Most cells would be zero (you never saw *jqxqz* in your data) so probabilities would be undefined or have to be smoothed into uselessness. **The counting approach fundamentally cannot scale.**

The curse of dimensionality, explained

What this table demonstrates is a fundamental statistical problem known as the **curse of dimensionality**. As the number of input dimensions grows, the volume of possible inputs grows exponentially, while the amount of training data available does not grow at all. With ten characters of context, there are roughly

10^{14} possible inputs but only about 10^5 examples in our dataset. The fraction of the input space that we actually observe during training is essentially zero, which means that for any reasonable test input, we have almost certainly never seen that exact context before. A counting model cannot assign a sensible probability to an unseen context because it has no count for it.

Why parametric models help

A neural network with a fixed parameter count produces predictions for *any* input — even ones it has never seen — by interpolating in a continuous representation space. Two contexts that differ slightly (say, the partial spelling *em* versus *en*) will produce similar internal representations, and therefore similar predictions, even if one context appeared a thousand times during training and the other appeared zero times. This is called **generalization**, and it is the central reason neural networks dominate modern machine learning. The bigram model has no mechanism for generalization — every context is treated as an independent symbol. The MLP we are about to build does, and that is the entire reason it works.

The pivot

We need a model that can take a longer context but whose number of parameters does *not* blow up exponentially with context length. The Bengio 2003 paper is the canonical answer: represent each token as a learned dense vector, concatenate those vectors, and feed them through a neural net. Parameter count then grows *linearly* with context length, not exponentially.

2. The Bengio 2003 paper at a glance

Chapter overview

This chapter introduces the paper whose architecture we are about to replicate. Understanding the Bengio 2003 paper is important for two reasons. First, it provides the architectural blueprint we will follow nearly verbatim in code. Second, the paper is one of the most historically important in modern NLP — it introduced the concept of learned word embeddings and represents the moment when neural networks started to overtake count-based methods in language modelling. The vocabulary and intuitions you build here will carry forward through the rest of the makemore series and into Transformers.

Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Jauvin published *A Neural Probabilistic Language Model* in JMLR 2003. It is the paper that introduced learned embeddings to language modelling, and the architectural blueprint we follow in this video is almost exactly theirs — just shrunk from words to characters.

Their architecture

- **Vocabulary:** 17,000 English words (we use 27 characters).
- **Embedding dimension:** 30 floats per word (we will try 2, then 10).
- **Context size:** 3 previous words (we will use 3 previous characters).

- **Hidden layer:** one fully-connected layer with tanh non-linearity, size is a hyperparameter (they tried a few; we will try 100, then 300, then 200).
- **Output layer:** 17,000-way softmax (we will have 27-way).
- **Loss:** “maximize the log-likelihood of the training data” — same objective as Part 1.

Why this paper was a milestone

Before Bengio 2003, the state of the art in language modelling was n-gram counting with elaborate smoothing schemes. Bengio's paper was the first to show that a neural network — trained end-to-end with backpropagation — could beat those classical methods on perplexity, while also producing side-products (the embeddings themselves) that turned out to be incredibly useful for downstream tasks. The embedding idea was then refined in Word2Vec (2013), GloVe (2014), and ELMo (2018), and eventually absorbed into modern Transformers as token embeddings. Every modern LLM still uses the exact same primitive Bengio introduced.

Why embeddings are the big idea

Bengio's key claim was that words with similar meanings should end up with similar embedding vectors — *sofa* and *chair* close together in the 30-D space, far from *airplane*. The embeddings are randomly initialized and then learned by backprop, alongside everything else. This means the model can generalize: even if it never saw “*the cat sat on the sofa*,” if it saw “*the cat sat on the chair*” and learned that sofa and chair are close, it can transfer.

For characters this matters less semantically (there is no meaningful difference between letters), but the machinery is identical. And it is the machinery that scales all the way to GPT's token embeddings.

How the architecture solves the curse of dimensionality

Notice the parameter count of Bengio's model: roughly (vocabulary × embedding-dim) for the embeddings, plus (context-size × embedding-dim × hidden-size) for the first weight matrix, plus (hidden-size × vocabulary) for the output. Critically, adding one more position to the context window only adds (embedding-dim × hidden-size) parameters to W_1 — a *linear* increase, not an exponential one. Compare this with the counting approach, where adding one position multiplies the table size by 27. This linear-versus-exponential difference is what makes the entire approach viable in practice.

PART II

Building the architecture

Cell by cell, we construct the MLP: embeddings, context windows, the flattening operation, the hidden layer, and the output.

3. Setup: imports, data, stoi and itos

Chapter overview

Every machine learning project begins with the same three setup steps: import the required libraries, load the data, and convert raw input into a format suitable for numerical processing. This chapter covers all three for the makemore project. Because we already did this work in Part 1, the cells are essentially copied forward — but it is worth understanding what each line is doing, because these same patterns recur in every subsequent project you will build.

The first few cells are essentially copied from Part 1. The vocabulary is unchanged (27 tokens), and `names.txt` is the same file.

Importing the required libraries

We will need three external libraries: PyTorch for tensors and automatic differentiation, the `functional` module of `torch.nn` for stateless neural network operations, and `matplotlib` for plotting. The import block below is standard for any PyTorch project and you should expect to see it at the top of essentially every notebook in this series.

Cell - Imports

```
import torch
from torch.nn import functional as F
import matplotlib.pyplot as plt
%matplotlib inline
```

`torch.nn.functional` is imported as `F` — this is the standard convention. Most of the building blocks we will use (`F.one_hot`, `F.cross_entropy`, `F.tanh`) live there.

What is `torch.nn.functional`?

PyTorch ships two parallel APIs for neural network operations: `torch.nn` contains stateful *modules* (such as `nn.Linear` or `nn.CrossEntropyLoss`) that hold their own parameters, while `torch.nn.functional` (typically aliased `F`) contains the same operations as *functions* with parameters passed in explicitly. For pedagogical purposes Karpathy uses the functional form throughout these lectures because it makes the math more visible. Production code often uses the module form.

Loading the dataset

The dataset is `names.txt`, a flat text file with one name per line. We read the whole file, split on newlines, and store the result as a Python list of strings.

Cells - Load words and build the character lookup

```
words = open('names.txt', 'r').read().splitlines()
print(words[:8])
len(words)           # 32033

chars = sorted(list(set(''.join(words))))
```

```
stoi = {s: i+1 for i, s in enumerate(chars)}
stoi['.'] = 0
itos = {i: s for s, i in stoi.items()}
```

Identical to Part 1. `stoi` maps each lowercase letter to an integer 1–26, with the special boundary token `.` at index 0. `itos` is the reverse.

Why we need an integer encoding

Neural networks operate on numbers, not text. Before any character can be fed into a model, it has to be assigned a unique integer. The two dictionaries `stoi` (string-to-integer) and `itos` (integer-to-string) implement this mapping. Their purpose is purely mechanical: convert symbols into a numerical form the model can consume, and convert the model's numerical predictions back into symbols when we sample from it. The convention of reserving index 0 for a special boundary token is what allows the model to learn how names begin and end.

4. Building the dataset with context windows

Chapter overview

Up to this point we still have raw strings; we now need to convert them into the matrix form a neural network actually consumes. Specifically, we need pairs of (context, target) examples, where the context is a fixed-length window of previous characters and the target is the next character. This chapter introduces the central concept of a **context window**, walks through the sliding-window construction, and verifies the resulting tensor shapes.

This is the first real architectural change. We are going to predict the next character from the previous `block_size` characters, so we need a dataset of (*context*, *target*) pairs.

What is a context window?

A context window is the fixed-size slice of recent tokens the model is allowed to look at when predicting the next one. In our model, with `block_size = 3`, the model can always see exactly the three most recent characters — never two, never four. The size of this window directly trades off model power (larger window = more information available) against parameter count and overfitting risk (larger window = more weights to learn). Modern Transformers use the same concept but with context windows of thousands of tokens; the underlying idea is identical.

Cell - Build (X, Y) with `block_size = 3`

```
block_size = 3                # context length: how many previous chars
X, Y = [], []

for w in words:
    context = [0] * block_size    # start with [., ., .]
    for ch in w + '.':
        ix = stoi[ch]
        X.append(context)
        Y.append(ix)
```

```
context = context[1:] + [ix] # slide the window
```

```
X = torch.tensor(X)
Y = torch.tensor(Y)
```

`block_size = 3` is the only hyperparameter that controls how much context the model sees. Three is what Bengio used; we will keep it here.

How the sliding window works

The dataset construction loop is the most procedurally dense block in the entire setup phase. Each name produces multiple training examples, one for each character (plus one for the terminating boundary token). The context window starts as three boundary tokens (representing the start of the name), and at each step we record the current context as an input, the current character as the target, and then slide the window forward by dropping the leftmost token and appending the current character on the right. This is the textbook implementation of an autoregressive language modelling dataset.

Tracing through *emma* with `block_size` 3:

Context	Target	Meaning
[. , . , .]	. e	name starts - predict 'e'
[. , . , e]	e m	after 'e', predict 'm'
[. , e , m]	m m	after 'em', predict 'm'
[e , m , m]	m a	after 'emm', predict 'a'
[m , m , a]	a .	after 'mma', predict end

- `context = [0] * block_size` — initialize with `[0, 0, 0]`, three copies of the boundary token `.`. This is how the model knows we are at the start of a word.
- `for ch in w + '.'` — iterate through each character of the word and the trailing end-token. Each name produces $\text{len}(\text{name}) + 1$ training examples.
- `context = context[1:] + [ix]` — slide the window: drop the leftmost element, append the newly-emitted character on the right. Crop and append.

Cell - Sanity-check shapes

```
X.shape, X.dtype, Y.shape, Y.dtype
# (torch.Size([32, 3]), torch.int64, torch.Size([32]), torch.int64)
# (using only words[:5] for the first experiments)
```

Initially Karpathy works with only the first 5 words to keep numbers small and printable. The first 5 names produce 32 (context, target) pairs — that is where the 32 comes from. Later, when we use all 32,000 words, the dataset grows to 228,146 examples.

5. The embedding matrix C

Chapter overview

This chapter introduces the most important new concept in Part 2: the **learned embedding**. Where Part 1 used a one-hot vector (sparse, mostly zeros, fixed) to represent each character, Part 2 uses a small dense vector of real numbers (typically 2 to 10 dimensions) that the network learns by gradient descent alongside everything else. This single change — sparse symbolic representation to learned dense representation — is the foundation of essentially every modern language model, from Word2Vec to GPT.

Each of the 27 characters needs to be turned into a vector before it can flow through the network. In Part 1 we used a one-hot vector (a 27-D vector with a single 1). That works but wastes 26 dimensions per character. Bengio's idea: learn a *dense* low-dimensional vector for each token. Two dimensions to start — small enough to plot.

What is an embedding?

An embedding is a learned function that maps a discrete token (a character, a word, a category label) to a continuous vector of real numbers. The key word is *learned*: the values in the vector are not fixed by us but are tuned by the optimizer to whatever makes the training loss low. After training, similar tokens (in whatever sense matters for the task) end up with similar vectors. This is in stark contrast to one-hot encoding, where every token is exactly the same distance from every other token and the model has no way to express the idea that two tokens are related.

Cell - Initialize the embedding matrix

```
C = torch.randn((27, 2))
```

C is a 27×2 tensor. Row i is the 2-D embedding vector for character i . At initialization the rows are just random normal samples; the network will tune them during training to whatever makes the loss low.

How the embedding is treated as a trainable parameter

The embedding matrix C is created using `torch.randn`, which fills it with samples from a standard normal distribution. The crucial detail is that C is just an ordinary tensor that we will later mark with `requires_grad=True`. PyTorch then automatically computes gradients with respect to C during backpropagation, and the optimizer updates its values alongside the weights W1, W2 and biases b1, b2. There is no special treatment for embeddings — they are simply another set of parameters in the model.

Why 2 dimensions?

Pedagogy. Karpathy picks 2 so we can plot the embeddings on a flat scatter plot at the end of the lecture and literally see whether vowels cluster together. For real models 2 is far too few — by the end of this notebook we will move to 10 dimensions, and modern LLMs use thousands.

6. Embedding lookup: indexing equals one-hot matmul

Chapter overview

Once the embedding matrix exists, we need a fast way to retrieve the row corresponding to a given character index. This chapter demonstrates three mathematically equivalent ways to do this and explains why we always prefer direct integer indexing in practice. The chapter also draws an important conceptual bridge: the embedding lookup is mathematically identical to the matrix multiplication we already did in Part 1, just written more efficiently. Recognizing this equivalence is what makes embeddings feel less like a new concept and more like a natural progression.

We need to convert an integer character index into its 2-D embedding vector. PyTorch makes this trivially easy. Three equivalent ways, demonstrated cell-by-cell:

Cell - Plain indexing

```
C[5]
# returns the 5th row of C - the 2-D embedding for character 'e'
```

Cell - Indexing with a tensor of indices

```
C[torch.tensor([5, 6, 7])]
# returns a (3, 2) tensor - three rows stacked
```

Cell - The one-hot route, for proof of equivalence

```
F.one_hot(torch.tensor(5), num_classes=27).float() @ C
# returns the same 2-D vector as C[5]
```

All three lines return the same vector. The third one is the formal “textbook” way: turn the index 5 into a 27-D one-hot vector, multiply by C. Because the one-hot vector has a 1 at position 5 and zeros elsewhere, the matmul plucks out row 5 of C. It is the same operation. The indexing form is just dramatically faster — no one-hot tensor is ever materialized.

Why integer indexing wins on performance

The reason direct indexing is used in practice is purely a matter of efficiency. The one-hot route requires materializing a 27-element vector (mostly zeros) for every character, then performing a full matrix multiplication where 26 of the 27 products are guaranteed to be zero. Integer indexing avoids both of these wastes: PyTorch goes straight to the memory location of the row we want and reads it directly. For our toy 27-character vocabulary the speed difference is negligible; for a Transformer with a vocabulary of 50,000 tokens the savings are enormous and indexing is the only way it would be feasible at all.

The hidden continuity from Part 1

In Part 1 we built a one-hot vector and matmul'd by W to get logits. That matmul was also “just a row lookup.” The MLP's embedding layer is exactly the same operation, just stripped of the fiction of a one-hot vector. Embeddings are not a new concept — they are the natural way to write what we were already doing.

7. Embedding the entire dataset in one operation

Chapter overview

PyTorch's indexing system is more powerful than ordinary Python indexing because it supports indexing with multi-dimensional integer tensors. This chapter shows how that capability lets us embed the entire dataset in a single operation, without writing any loops. The resulting three-dimensional tensor is what flows into the rest of the network.

The same indexing magic works with multi-dimensional integer tensors. We can embed every character in every context of the entire dataset in a single line:

Cell - Embed all of X

```
C[X].shape
# torch.Size([32, 3, 2])
```

X has shape (32, 3) — 32 examples, 3 characters of context each. C[X] has shape (32, 3, 2) — for every example and every context position, we now have a 2-D embedding vector. PyTorch broadcasts the indexing operation across all dimensions of X.

How does C[X] work mechanically?

X is a tensor of shape (32, 3) containing integer indices in the range [0, 27). When we write C[X], PyTorch treats every element of X as an index into the first dimension of C. The output has shape (32, 3, 2) because PyTorch replaces each scalar index in X with the corresponding 2-dimensional row from C. This is called *advanced indexing*, and it is one of the most powerful features of NumPy and PyTorch.

Cell - Verify the lookup did what we think

```
X[13, 2]          # the integer at row 13, column 2 of X
C[X][13, 2]       # the embedding vector at that position
C[1]              # the embedding for character index 1
# if X[13, 2] == 1, then C[X][13, 2] == C[1]
```

Cell - Stash it

```
emb = C[X]
emb.shape      # (32, 3, 2)
```

emb is now a 3-D tensor we will feed into the rest of the network. The comment in your notebook captures it well: “PyTorch embedding is awesome to embed everything simultaneously.”

8. The hidden layer: W1, b1

Chapter overview

After embeddings, the next layer of the MLP is a fully-connected linear transformation followed by a non-linearity. This chapter introduces the weight matrix W1 and bias vector b1 that parameterize this layer, and explains why the dimensions are what they are. The dimensions are not arbitrary — they follow directly from the previous design choices (block size, embedding dim) and from the hidden layer width, which is the one new hyperparameter introduced in this chapter.

After the embedding layer, Bengio's architecture has a single fully-connected hidden layer with a tanh non-linearity. We set up its parameters:

Cell - Initialize hidden-layer parameters

```
w1 = torch.randn((6, 100))    # 3 * 2 = 6 inputs, 100 hidden neurons
b1 = torch.randn(100)         # one bias per neuron
```

Why (6, 100)?

- **6 input dimensions:** we have `block_size = 3` context positions, each turned into a 2-D embedding. So the concatenated input to this layer is $3 \times 2 = 6$ numbers.
- **100 output dimensions:** this is the hidden-layer width, a hyperparameter chosen by us. Bigger = more capacity, more parameters, slower training, easier to overfit. Karpathy starts at 100, later bumps it to 300, then settles on 200 once embeddings are also bigger.
- **The bias vector `b1` has shape (100,)** — one bias for each hidden neuron, broadcast across the batch dimension during the forward pass.

Why the dimensions chain together this way

The shape of every weight matrix in the network is dictated by the shape of the input it receives and the shape of the output it must produce. Here, the input is a (batch, 6) tensor (after flattening) and we want an output of shape (batch, 100). The weight matrix multiplying these must therefore be (6, 100). The bias is added element-wise across the second dimension, so it has shape (100,). This dimensional chain — from input shape, through every layer, to output shape — is the single most common source of bugs when writing networks from scratch. Get into the habit of annotating every tensor with its expected shape; you will save yourself hours.

9. The flattening problem - (32, 3, 2) is not (32, 6)

Chapter overview

We now run into the first real technical obstacle: the shape of `emb` is three-dimensional, but the matrix multiplication we want to perform requires a two-dimensional input. This chapter explains what the obstacle is, why it exists, and sets up the three solutions that the next two chapters work through in detail. This kind of shape-juggling is bread-and-butter work in deep learning, and the techniques covered here recur constantly.

Here is the snag. `emb` has shape (32, 3, 2), but to multiply it by `w1` of shape (6, 100) we need the input to be shape (32, 6). Matmul rules do not let us multiply a 3-D tensor by a 2-D tensor in the way we want.

Cell - Try the naive thing (this almost works, but not quite)

```
emb @ w1 + b1
# RuntimeError or unexpected broadcasting - the shapes do not line up
```

We need to *flatten* the middle two dimensions: combine the “3 context positions” and “2 embedding dims per position” into a single “6 numbers per example” dimension. PyTorch gives us several ways to do this,

and the lecture walks through each one to teach the API.

Why we need to flatten

Mathematically, what we want is to treat the three context positions as a single feature vector of length six. In other words, the model should learn weights that look at all three context positions simultaneously, with no architectural distinction between “position 0” and “position 2” — that distinction lives in the weights, which can learn to attend more to recent positions if that helps. Flattening makes this concatenation explicit so that the matrix multiplication treats the six numbers as one input vector.

10. Three ways to flatten: cat,.unbind+cat, .view

Chapter overview

PyTorch provides several routes to reshape a tensor, and Karpathy walks through three of them in order from clumsy to elegant. Each method produces the same result; they differ only in code readability and runtime performance. Working through all three is a good way to learn the relevant API surface and to internalize the difference between operations that copy memory and operations that merely relabel it.

Way 1 - torch.cat with hard-coded indices

```
torch.cat([emb[:, 0, :], emb[:, 1, :], emb[:, 2, :]], 1).shape
# torch.Size([32, 6])
```

We manually grab each context position — `emb[:, 0, :]` is the embedding for the first context position across all 32 examples, shape `(32, 2)` — and concatenate them along dimension 1. Works, but ugly: change `block_size` from 3 to 5 and you have to rewrite this line.

Way 2 - torch.unbind

```
torch.cat(torch.unbind(emb, 1), 1).shape
# torch.Size([32, 6])
```

`torch.unbind(emb, 1)` removes dimension 1 and returns a tuple of slices: `(emb[:,0,:], emb[:,1,:], emb[:,2,:])`. We cat that tuple along dim 1. This generalizes — does not matter if `block_size` is 3 or 5 or 50. But it still allocates new memory and copies data around.

Way 3 - .view (the winner)

```
emb.view(32, 6)          # explicit
emb.view(-1, 6)          # let PyTorch infer the batch dimension
```

`.view` reinterprets the same underlying memory as a different shape, with no copy. Your notebook verifies the equivalence:

```
emb.view(32, 6) == torch.cat(torch.unbind(emb, 1), 1)
# tensor of all True
```

Why all three produce identical results

The output is identical in every case because all three methods are computing the same logical operation: take the (32, 3, 2) tensor and rearrange its 192 values into a (32, 6) layout where the six values for each example are the concatenation of its three 2-D embeddings. The difference is entirely in *how* the rearrangement is done. `cat` and `unbind+cat` physically allocate new memory and copy values; `.view` changes only the shape metadata. The next chapter explains why that distinction matters.

11. Why `.view` is the winner

Chapter overview

This chapter takes a brief detour into PyTorch internals to explain what actually happens when you call `.view`. The takeaway is that `.view` is essentially free at runtime, so you should prefer it whenever it is applicable. Understanding the memory model behind tensors will also help you debug shape errors more quickly later in your career.

PyTorch tensors have two layers: the data itself (a flat 1-D block of memory) and the metadata describing how to interpret that block as a multi-dimensional array. Shape, stride, offset — that metadata. `.view` changes only the metadata. No data is moved, no memory is allocated. It is effectively free.

Tensors as storage plus metadata

Every PyTorch tensor consists of a contiguous one-dimensional buffer of memory called *storage*, plus a small piece of metadata describing how to interpret that buffer (shape, stride, offset, dtype). Two tensors can share the same underlying storage while having different shapes and strides — this is exactly what `.view` exploits. Operations like `cat` have to allocate fresh storage because they combine data from separate buffers; `.view` simply creates a new metadata header pointing at the existing buffer.

Reading tip from Karpathy

He says: “You should read this excellent blog post by Edward Yang from PyTorch — ‘PyTorch internals.’ It explains exactly how the storage and the view-metadata interact.” This is the reason `.view` is so much faster than `cat` or `unbind`: it is not a real operation, it is a relabelling.

Two more idioms worth memorizing:

- **The -1 trick.** `emb.view(-1, 6)` tells PyTorch “figure out this dimension from the total number of elements.” You only get to use `-1` once per call. Use it for the batch dimension so the code works for any batch size.
- **`.view` vs `.reshape`.** They look similar; `.view` requires the tensor to be contiguous in memory and is always a no-copy, while `.reshape` falls back to a copy if needed. Karpathy uses `.view` because he wants the no-copy guarantee, and the tensors here always happen to be contiguous.

Cell - Wire up the hidden activations

```
h = torch.tanh(emb.view(-1, 6) @ w1 + b1)
```

`h.shape` `# (32, 100)` - 100-D activations for each of 32 examples

12. The tanh non-linearity

Chapter overview

Linear layers alone, no matter how many you stack, can only represent linear functions of the input. To make the network actually *deep* — meaning capable of representing non-linear functions of arbitrary complexity — we need a non-linearity between every pair of linear layers. This chapter introduces `tanh`, the non-linearity used in the original Bengio paper and what we will use here. It also briefly contrasts `tanh` with the modern default (ReLU) so you understand the tradeoffs.

`torch.tanh` applies the hyperbolic tangent element-wise. It squashes any real number into the range $(-1, 1)$: big positive numbers saturate near $+1$, big negative numbers saturate near -1 , and small values pass through nearly linearly.

Why we need a non-linearity at all

If we stacked two linear layers without anything between them, the composition $(x @ W_1 + b_1) @ W_2 + b_2$ is mathematically equivalent to a single linear layer with weight $W_1 @ W_2$. No matter how deep we stack, the network can only represent linear functions of the input. **The non-linearity is what makes “deep” deep.**

The universal approximation theorem

There is a classical result in neural network theory called the universal approximation theorem, which states that a feedforward network with at least one hidden layer and a non-linear activation function can approximate any continuous function arbitrarily well, given enough neurons. The theorem does not tell us anything about how easy it is to *find* such an approximation by gradient descent, but it does explain why even a one-hidden-layer MLP is more expressive than a linear model: the non-linearity is what unlocks that expressive power.

Why tanh specifically

`tanh` was the default in 2003 because it is smooth (differentiable everywhere) and zero-centered. Modern nets typically use ReLU and its variants, which are cheaper to compute and avoid the “vanishing gradient” problem that haunts deep tanh networks (Karpathy spends time on this in Part 3, where we deliberately stress-test the initialization). For Bengio’s shallow MLP, `tanh` is fine.

13. The output layer and logits

Chapter overview

The final layer of the network takes the 100-dimensional hidden activations and projects them to 27 raw scores, one per possible next character. These scores are called *logits*, and this chapter explains what

they mean, why we do not yet apply a softmax to them (we will, but only inside `F.cross_entropy` later), and how the dimensions chain together.

Cell - Initialize the output layer

```
w2 = torch.randn((100, 27))  
b2 = torch.randn(27)
```

The output is a 27-way classification — one score per possible next character. So `w2` is shape `(100, 27)` (100 hidden units in, 27 outputs) and `b2` is shape `(27, )`.

Cell - Compute the logits

```
logits = h @ w2 + b2  
logits.shape  # (32, 27)
```

“Logits” is the standard name for the raw, pre-softmax output of a classifier. Same interpretation as Part 1: these are *log-counts*. The next two lines turn them into probabilities.

What are logits and why do we work with them?

A *logit* is a real-valued score assigned by the classifier to each possible output class, before any normalization is applied. Logits can be any real number (positive, negative, large, or small). To turn them into a probability distribution, we apply softmax, which exponentiates and normalizes. But many modern training objectives — including cross-entropy — are most numerically stable when they receive logits directly rather than probabilities, because they can fold the softmax into their internal computation. This is why `F.cross_entropy` expects logits, not probabilities.

PART III

Training the model

Loss functions, the training loop, minibatches, and the most important hyperparameter in deep learning: the learning rate.

14. Manual softmax and NLL - the slow way

Chapter overview

Before we adopt the fused `F.cross_entropy` function in Chapter 16, Karpathy first builds the loss by hand using primitive operations. This exercise is purely pedagogical — it makes the math visible — but it is the single best way to understand what cross-entropy actually computes. Read this chapter carefully even though we will throw the code away in two chapters' time.

Just to show the equivalence to Part 1, the notebook first does the softmax + NLL by hand:

Cell - Manual softmax

```
counts = logits.exp()                # exp turns log-counts into counts
prob = counts / counts.sum(1, keepdims=True)  # row-normalize -> probabilities
prob.shape                          # (32, 27)
```

How softmax converts scores into a probability distribution

The softmax function works in two steps. First, every logit is exponentiated, which converts the real-valued scores into positive numbers. Larger logits become much larger after exponentiation; smaller logits become much smaller. Second, every row is normalized by dividing by the sum of its exponentiated values. After normalization, each row sums to exactly 1.0 and every value is in the range (0, 1), so each row is a valid probability distribution over the 27 possible next characters. The `keepdims=True` argument preserves the second dimension after summing, which makes the subsequent broadcast division work correctly without manual reshaping.

Cell - Pluck out the probability assigned to the correct next char

```
Y                                # the 32 correct next-character indices
torch.arange(32)                 # 0, 1, 2, ..., 31
loss = -prob[torch.arange(32), Y].log().mean()
loss                            # tensor(17.7697)
```

Exactly the same vectorized NLL line from Part 1. For each of the 32 examples, look up `prob[i, Y[i]]` — the probability the network assigned to the correct answer — take its log, average, negate.

Why negative log-likelihood is the right objective

The reason we minimize negative log-likelihood rather than, say, raw probability or accuracy comes down to a principle from information theory called *maximum likelihood estimation*. If we assume our model is producing the true distribution of next characters, then maximizing the joint probability of observing the training set is equivalent to minimizing the average negative log of the predicted probabilities for the observed characters. Equivalently, the loss is the average number of bits (in natural log units, called *nats*) the model would need to compress each character given its context. Lower loss means more compression, which means better prediction.

Why is the initial loss 17.77 and not $\log(27) \sim 3.30$?

If the network were outputting a perfectly uniform distribution, NLL would be $\log(27) \sim 3.30$ — same as Part 1's untrained baseline. We get 17.77 because the random initialization happens to assign *very high* probability to *wrong* characters and *tiny* probability to the correct ones. The logits are scattered with large magnitudes, so after softmax the distribution is sharp and confidently wrong. The first few gradient steps will fix this dramatically — loss will fall from 17 to around 3 almost instantly. (Part 3 addresses this initialization problem properly.)

15. Making it respectable - parameter lists and generators

Chapter overview

Karpathy now reorganizes the prototype code into a cleaner, more maintainable form before scaling up. The two changes — deterministic initialization via a generator and collecting parameters into a list — are not just cosmetic. They establish patterns you will use in every PyTorch project you write from this point forward, and they correspond directly to features that PyTorch's `nn.Module` framework wraps later in the series.

Karpathy now consolidates the prototype into clean code. Two new conventions:

Cell - Reproducible initialization with a generator

```
g = torch.Generator().manual_seed(2147483647)
C = torch.randn((27, 2), generator=g)
W1 = torch.randn((6, 100), generator=g)
b1 = torch.randn(100, generator=g)
W2 = torch.randn((100, 27), generator=g)
b2 = torch.randn(27, generator=g)
parameters = [C, W1, b1, W2, b2]
```

- **Pass the generator to every random op.** Without this, every cell re-run gives different numbers and you cannot reproduce results. 2147483647 is the same seed Karpathy used in Part 1 (it is $2^{31} - 1$).
- **Cluster parameters into a list.** Now we can iterate over `parameters` when zeroing gradients or applying updates — one loop handles all five tensors. This is the precursor to PyTorch's `nn.Module.parameters()` idiom.

Why reproducibility matters

Deterministic initialization is a fundamental requirement of scientific machine learning. Without it, you cannot tell whether a change in your loss came from a real improvement in the code or from random variation in the starting weights. Always seed your random number generators before doing anything important. PyTorch provides three separate sources of randomness (`torch.Generator`, the global PyTorch seed via `torch.manual_seed`, and Python's own `random` module) and you may need to seed

all three depending on what your code does.

Cell - Count total parameters

```
sum(p.nelement() for p in parameters)
# 3481 - about 3.5k parameters
```

`.nelement()` returns the number of elements in a tensor. Summing across all parameters gives the total parameter count of the model. 3.5k for the first version — tiny. By the end of the notebook we will be around 12k.

16. F.cross_entropy - efficiency and numerical stability

Chapter overview

This chapter introduces the function you will use in essentially every classification model you build from this point on: `F.cross_entropy`. It is a single drop-in replacement for the entire softmax+log+NLL pipeline we built by hand in Chapter 14. The reasons to prefer it are concrete and important: it is faster, it uses less memory, and most critically it is *numerically stable* in situations where the manual implementation silently corrupts your training. This chapter explains all three reasons in detail.

Cell - Replace four lines with one

```
# Before: manual softmax + NLL
# counts = logits.exp()
# prob = counts / counts.sum(1, keepdims=True)
# loss = -prob[torch.arange(32), Y].log().mean()
```

```
# After:
F.cross_entropy(logits, Y)
```

`F.cross_entropy` takes raw logits (not probabilities!) and integer targets, and returns the average NLL. One call replaces the whole softmax-then-log pipeline. There are three reasons to always prefer it over rolling your own:

Reason 1 - Fused kernel (speed and memory)

Our manual version creates three intermediate tensors: counts, prob, and the gathered probabilities. Each one allocates memory. `F.cross_entropy` fuses these into a single CUDA kernel that streams through the data once. No intermediates, much less memory pressure, much faster.

Reason 2 - Simpler backward pass

The forward pass of softmax+log+gather is complicated. Backpropping through each step would compute gradients for the intermediates we never need. But mathematically, the gradient of cross-entropy with respect to logits simplifies to a beautiful one-liner: `softmax(logits) - one_hot(Y)`. PyTorch implements this analytical form directly, skipping all the intermediate gradient computations. Just like the 1 - tanh² trick from micrograd.

Reason 3 - Numerical stability

This is the killer reason. Imagine during training a logit grows to +100. Then $\exp(100) \approx 2.7 \times 10^{43}$, which overflows float32 and becomes `inf`. Dividing `inf` by `inf` gives `nan`, and your training collapses irrecoverably.

`F.cross_entropy` uses the “log-sum-exp” trick: before exponentiating, it subtracts the max logit from every logit. Mathematically this does not change the softmax (the same constant cancels in the numerator and denominator), but it caps the largest exponent at $\exp(0) = 1$, so nothing can overflow. Negative logits can still go to $\exp(-\infty) = 0$, but that is harmless — we just get a tiny probability, not a NaN.

The log-sum-exp trick illustrated

Suppose logits are [2, 3, 100]. Naively exponentiating gives $[e^2, e^3, e^{100}]$, where the last value overflows. Subtracting the max (100) first gives [-98, -97, 0], which exponentiates to $[e^{-98}, e^{-97}, 1]$. The first two values are tiny but representable, and after normalization the resulting probabilities are identical to what we would have computed without the subtraction. This same trick is used in every modern softmax implementation.

Rule of thumb

Whenever PyTorch ships a fused function for something you would implement by hand (cross-entropy, log-softmax, layer norm, attention), use it. The fused version will be faster, more memory-efficient, and numerically safer. The only exception is when you are learning — like in this notebook — and want to see the underlying math.

17. The first training loop - overfitting on purpose

Chapter overview

We now have everything required for a forward pass: data, parameters, a forward function, and a loss. This chapter assembles the pieces into a complete training loop and runs it on a deliberately tiny subset of the data — just the 32 examples from the first 5 words. The reason for using such a small dataset is methodological: if the network has enough capacity to perfectly memorize 32 examples, we have proved that the forward pass, backward pass, and parameter updates are all wired up correctly. This is a standard debugging technique that you will use throughout your career.

Before scaling to the full dataset, Karpathy trains on just the 32 examples we have (from the first 5 words) to prove the optimization machinery works.

Cell - Enable gradients on all parameters

```
for p in parameters:
    p.requires_grad = True
```

What does `requires_grad = True` do?

Setting `requires_grad = True` on a tensor tells PyTorch to track every operation that uses that tensor in a directed acyclic graph (the computation graph). When you subsequently call `loss.backward()`, PyTorch walks that graph in reverse and accumulates the gradient of the loss with respect to every tracked tensor into its `.grad` attribute. Without this flag, tensors are treated as constants during backpropagation and no gradients are computed for them.

Cell - The training loop

```
for i in range(1000):
    # forward
    emb = C[X]                                # (32, 3, 2)
    h = torch.tanh(emb.view(-1, 6) @ W1 + b1)  # (32, 100)
    logits = h @ W2 + b2                      # (32, 27)
    loss = F.cross_entropy(logits, Y)

    # backward
    for p in parameters:
        p.grad = None
    loss.backward()

    # update
    for p in parameters:
        p.data += -0.1 * p.grad

print(loss.item())
# something tiny, like 0.25
```

The pattern is the same as Part 1, generalized to multiple parameter tensors:

- Forward pass — embed, hidden layer, output, loss.
- Backward pass — clear gradients (`p.grad = None`), then call `loss.backward()`. PyTorch walks the graph from loss back to every parameter with `requires_grad=True`.
- Update — nudge each parameter in the direction opposite its gradient. Learning rate 0.1, picked by gut.

How the three phases of the training loop work together

Every training loop in deep learning follows this same forward-backward-update pattern. The forward pass computes the loss as a function of the current parameters. The backward pass uses the chain rule (implemented as `loss.backward()`) to compute the gradient of the loss with respect to every parameter, populating their `.grad` fields. The update step then takes a small step in the direction that decreases the loss, namely the opposite of the gradient. Setting `p.grad = None` before the backward pass is critical because PyTorch accumulates gradients by default — without zeroing, gradients from previous iterations would still be present and corrupt the update.

Why does Karpathy deliberately overfit 32 examples?

It is a sanity check that the network has enough capacity to fit something. 3481 parameters trained on 32 examples can memorize them — loss should drop to near zero. If it cannot even overfit a tiny dataset, there is a bug in your forward pass. This is a debugging move every ML practitioner does.

Why loss does not go to exactly zero

Loss settles around 0.25, not 0.0. The reason: in the first 5 words, the context [. , . , .] appears five times — once for each word's first character. But the “correct” next character differs: e for emma, o for olivia, a for ava, i for isabella, s for sophia. The same input has five different targets, so no setting of the weights can give probability 1 to all five. The lowest the loss can go on those examples is the entropy of that target distribution. The remaining examples drag the average loss down to 0.25.

18. Scaling up: full dataset of 228k bigrams

Chapter overview

Having validated the training machinery on a tiny dataset, we now scale up to all 32,000 names, which produces 228,146 (context, target) training examples. The model architecture is unchanged. The training loop is unchanged. What changes is performance: with hundreds of thousands of examples to forward and backward through on every iteration, each gradient step now takes a noticeable amount of time. This sets up the need for the minibatch trick introduced in the next chapter.

Rebuild the dataset using all of words, not just the first 5. The dataset grows from 32 to 228,146 examples. Same model, same training loop — we just run it on the full data.

The first issue: speed. Every iteration now embeds, matmuls, and backprops on 228k examples. That is slow.

19. Minibatches with torch.randint

Chapter overview

The single most important performance trick in modern deep learning is the minibatch: at every training step, randomly sample a small subset of the data, compute the gradient on just that subset, and take a step. The gradient computed on a minibatch is a noisy estimate of the true full-dataset gradient, but it is dramatically cheaper to compute, and many noisy steps generally beat few exact ones. This chapter introduces the technique and explains why it works.

The fix is one of the load-bearing tricks of deep learning: do not compute the gradient on the entire dataset. Sample a small random subset (a **minibatch**), compute the gradient on just that, take a step, repeat.

What is a minibatch?

A minibatch is a small random subset of the training data — typically between 32 and 512 examples — used to compute a single gradient step. Each training step uses a fresh minibatch, drawn uniformly at random (usually with replacement, for simplicity). Over many steps, every example in the dataset is seen many times. The gradient computed on the minibatch is a noisy but unbiased estimate of the gradient on the full dataset, which means on average the minibatch update points in the same direction as the full-batch update.

Cell - Sample a minibatch

```
ix = torch.randint(0, X.shape[0], (32,))  
# 32 random integer indices into the dataset, with replacement
```

- `torch.randint(0, N, (32,))` returns 32 random integers in the range `[0, N)`. These are the indices of the examples we will use this step.
- `X[ix]` grabs those 32 rows from `X`; `Y[ix]` grabs the corresponding targets.
- **The forward pass is identical** — just on the 32-row subset instead of the full 228k.

In your notebook this becomes the standard form of the inner loop:

```
ix = torch.randint(0, X.shape[0], (32,))  
emb = C[X[ix]] # (32, 3, 2)  
h = torch.tanh(emb.view(-1, 6) @ W1 + b1)  
logits = h @ W2 + b2  
loss = F.cross_entropy(logits, Y[ix])  
  
for p in parameters: p.grad = None  
loss.backward()  
for p in parameters: p.data += -lr * p.grad
```

Why noisy gradients still produce good models

Intuitively you might worry that a noisy gradient direction would cause the optimizer to wander aimlessly. In practice, the opposite happens. The minibatch gradient is, in expectation, equal to the full-batch gradient — so on average each step is correct. The noise around that average has two useful side effects: it lets the optimizer escape narrow local minima that full-batch SGD would get stuck in, and it acts as a mild form of regularization that improves generalization to unseen data. This is why minibatch SGD is the standard, even when full-batch computation is feasible.

Approximate gradient on 32 examples vs exact gradient on 228k

“The gradient direction is good enough, even when it is estimated on only 32 examples, that it is useful. It is much better to have an approximate gradient and make many steps than it is to evaluate the exact gradient and make few steps.” — Karpathy. The minibatch direction is noisy, but cheap. Many noisy steps beat few precise ones. Your notebook puts it concisely: *approx direction and take steps >>> exact direction and take the steps*.

20. The learning rate finder

Chapter overview

The learning rate is the single most important hyperparameter in deep learning. Set it too low and training is glacially slow; set it too high and the loss explodes. Until now we have been guessing it. This chapter introduces a clever empirical procedure for *finding* a good learning rate instead of guessing. The technique is fast, foolproof, and applicable to essentially any model.

We have been guessing learning rates. Karpathy now shows a clever empirical trick to *find* a good one. The idea: sweep through many candidate learning rates exponentially, plot loss as a function of learning rate, and read off the value where loss is still falling fast but not yet exploding.

Cell - Build a logarithmically-spaced range of learning rates

```
lre = torch.linspace(-3, 0, 1000)
lrs = 10**lre
```

Reading this carefully:

- `torch.linspace(-3, 0, 1000)` gives 1000 numbers evenly spaced between -3 and 0 . Call these the “learning rate *exponents*.”
- `10**lre` raises 10 to each one, giving learning rates that are exponentially spaced between $10^{-3} = 0.001$ and $10^0 = 1$.
- **Why exponentially spaced?** Because intuition about learning rates is multiplicative, not additive. The interval $[0.001, 0.01]$ is the same kind of jump as $[0.01, 0.1]$ — both $10\times$. Linear spacing would waste most of the candidates near 1.0 .

How the sweep is implemented

The learning rate sweep is implemented by running 1000 training iterations where each iteration uses a different learning rate, ramping from 0.001 at the start to 1.0 at the end. At each step we record the learning rate used and the loss it produced. Crucially, the sweep is fast because we are using minibatches — the entire 1000-step sweep takes a few seconds. After the sweep finishes, we plot loss as a function of learning rate and visually identify the regime where loss falls fastest.

Cell - Sweep through the learning rates while training

```
lri, lossi = [], []

for i in range(1000):
    ix = torch.randint(0, X.shape[0], (32,))

    # forward
    emb = C[X[ix]]
    h = torch.tanh(emb.view(-1, 6) @ W1 + b1)
    logits = h @ W2 + b2
    loss = F.cross_entropy(logits, Y[ix])

    # backward
    for p in parameters: p.grad = None
```

```

loss.backward()

# update - using a DIFFERENT learning rate each step
lr = lrs[i]
for p in parameters:
    p.data += -lr * p.grad

# log the LR and the loss it produced
lri.append(lre[i])          # exponent, for nicer plotting
lossi.append(loss.item())

```

Step i uses learning rate $lrs[i]$. Step 0 uses 0.001 (basically does nothing), step 999 uses 1.0 (probably explodes). We track each LR exponent and the resulting loss.

21. Reading the LR-vs-loss plot - choosing 0.1

Chapter overview

With the sweep complete, we now interpret the resulting plot to decide on a good learning rate. The shape of the curve is consistent across essentially all models — once you have seen one of these plots you will recognize the structure immediately. This chapter walks through how to read the plot and arrive at a concrete recommendation.

Cell - Plot

```
plt.plot(lri, lossi)
```

You see a curve with three regions:

- **Left side (very small LR, exponent near -3):** loss barely moves. The model is being nudged so gently it cannot escape its initial bad guess.
- **Middle (exponents around -1):** loss falls steeply. This is the sweet spot.
- **Right side (exponent near 0, LR near 1):** loss is chaotic, jumping around, or rising. The updates are so large they overshoot the loss landscape.

The right learning rate is in the middle of the “loss falling steeply” region. From the plot, the exponent is around -1, which means $LR \approx 10^{-1} = 0.1$. Your notebook records this finding: *“as we can see -1 i.e. -0.1 was a very good lr, which we are gonna keep.”*

The LR-finder technique in production

This trick is sometimes called the “Leslie Smith LR finder” (introduced in a 2015 paper and popularized by fast.ai). For a tiny model like ours, 1000 sweep steps takes seconds. For a large transformer it would take longer, but the technique scales fine if you sweep over a few hundred steps with a small batch. Beats guessing every time.

22. Learning rate decay

Chapter overview

A constant learning rate is fine for the early stages of training, when the model is far from the minimum and needs to take large steps to make rapid progress. But near the end of training, when the loss is close to its minimum value, a large learning rate causes the optimizer to overshoot and oscillate around the minimum without settling. The fix is learning rate decay: lower the learning rate as training progresses, so that the optimizer takes smaller and smaller steps near the end. This chapter introduces the simplest form of decay — a single step-down at the end — and explains why it works.

Train with LR=0.1 for many thousand steps and the loss falls quickly, then starts to oscillate around some plateau. The fix is the second big LR trick: **learning rate decay**. After most of the descent is done, drop the learning rate by 10× (to 0.01) and run a few more thousand steps. The smaller step size lets the optimizer settle into a better minimum it was previously overshooting.

Cell - Decay at the end

```
# during the final phase of training:
lr = 0.01    # was 0.1
for p in parameters:
    p.data += -lr * p.grad
```

Your notebook captures the intuition: “*at the last of the training stages make 0.1 to 0.01, just to slow down at the end as we near the min loss.*” Production training uses more sophisticated schedules (cosine, linear warmup + decay, ...), but the principle is the same: large steps to find the basin, small steps to settle in it.

Common learning rate schedules

Beyond the simple step-down used here, production training pipelines typically use one of several smoother schedules. *Cosine annealing* smoothly reduces the learning rate following a cosine curve from the initial value down to a small final value. *Linear warmup + linear decay* starts at zero, ramps up linearly to the peak learning rate over a few hundred steps, then decays linearly back down. *Cyclic schedules* oscillate the learning rate between bounds. All of these follow the same underlying principle: large steps early, small steps late, with a smooth transition in between.

PART IV

Diagnostics and scaling

The train/dev/test discipline, detecting underfitting and overfitting, scaling up the model intelligently, and the embedding visualization.

23. The train / dev / test split - what each one is for

Chapter overview

Up to this point we have been training and evaluating on the same data, which is methodologically indefensible: a model that has memorized the training set will report a flatteringly low loss on it, but that loss tells us nothing about how the model will perform on inputs it has never seen. This chapter introduces the discipline of splitting your data into three disjoint subsets — train, dev, and test — each with a different, non-overlapping role. Once internalized, this discipline becomes second nature for every machine learning project you build.

At this point the notebook hits its first real plateau and Karpathy delivers a key methodological lesson. Your notebook frames the problem perfectly:

The problem you wrote down

“Since dataset is small, it is memorising it, which leads to overfitting. We get to know that when we try to sample datasets from it, we only get the same examples which were provided to you. To deal with this we split our dataset into training set, dev/validation set (used over the hyperparameters of the model) and test set.”

The three splits and their disjoint jobs:

Split	Size	Used for
Train	80%	Updating parameters with gradient descent.
Dev (validation)	10%	Comparing different models and hyperparameters (LR, hidden size, embedding dim, block size). You touch this many times during development — that is fine.
Test	10%	Final, single-use evaluation. If you tune on this, you have leaked the test set into your decisions. Touch it once, at the very end, after all hyperparameters are frozen.

Why three splits and not just two

A common beginner mistake is to use only two splits, training and test, and to repeatedly evaluate on the test set while tuning hyperparameters. This is statistically equivalent to training on the test set, just laundered through human decision-making: every time you compare two settings on the test set, you are extracting information from it and using that information to choose your model. After enough comparisons, your test score becomes optimistic and no longer reflects how the model will perform in the real world. The dev set is the “sandbox” where you do that experimentation; the test set is reserved for a single, honest final report.

[Cell - Build the splits](#)

```
def build_dataset(words):
    block_size = 3
    X, Y = [], []
    for w in words:
        context = [0] * block_size
        for ch in w + '.':
            ix = stoi[ch]
            X.append(context)
            Y.append(ix)
            context = context[1:] + [ix]
    X = torch.tensor(X)
    Y = torch.tensor(Y)
    print(X.shape, Y.shape)
    return X, Y

import random
random.seed(42)
random.shuffle(words)
n1 = int(0.8 * len(words))
n2 = int(0.9 * len(words))

Xtr, Ytr = build_dataset(words[:n1])      # 80% - train
Xdev, Ydev = build_dataset(words[n1:n2])  # 10% - dev
Xte, Yte = build_dataset(words[n2:])      # 10% - test
```

- `random.seed(42)` — seed Python's PRNG (separate from PyTorch's) so the shuffle is reproducible. 42 is a famous arbitrary number from Hitchhiker's Guide; people use it everywhere.
- `random.shuffle(words)` — modifies words in place. Critical: the file was sorted by name frequency. Without shuffling, your train set would be common names and your test set would be rare names — a distribution shift that would make dev/test loss artificially high.
- **The function** `build_dataset` — same windowing logic as before, factored into a function so we can call it on each split.

24. Underfitting: when train loss equals dev loss

Chapter overview

Once the train/dev/test discipline is in place, comparing the loss on train and dev becomes the single most informative diagnostic in your toolkit. The relative magnitudes of these two losses tell you immediately whether your model has too little capacity (underfitting), too much capacity for the amount of data (overfitting), or is well-matched to the data (the healthy regime). This chapter introduces the three scenarios as standalone diagnostic cards and explains how to respond to each.

After training the model on the train split, we evaluate on both train and dev:

Cell - Evaluate on train

```
emb = C[Xtr]
h = torch.tanh(emb.view(-1, 6) @ W1 + b1)
logits = h @ W2 + b2
```

```
loss = F.cross_entropy(logits, Ytr)
loss  # something like 2.30
```

Cell - Evaluate on dev

```
emb = C[Xdev]
h = torch.tanh(emb.view(-1, 6) @ W1 + b1)
logits = h @ W2 + b2
loss = F.cross_entropy(logits, Ydev)
loss  # also about 2.30
```

Reading the train/dev comparison

There are three scenarios you might observe when you compare train loss to dev loss. Each one points at a specific underlying problem and prescribes a specific corrective action. The diagnostic cards below summarize all three. Internalize them: you will use this framework constantly.

Scenario 1 · Underfitting	
Symptom	Train loss and dev loss are both still high, and they are approximately equal to each other.
Diagnosis	The model is too small for the data. It cannot even fit the training set well, so it has no chance of doing better on examples it has never seen. The architecture is the bottleneck.
Action	Make the model bigger. Increase the hidden layer width, increase the embedding dimension, or add more layers. Add capacity until the train loss starts to drop below the dev loss.

Scenario 2 · Overfitting	
Symptom	Train loss is low (often very low), but dev loss is much higher than train loss and may even be rising over time.
Diagnosis	The model has memorized specific training examples instead of learning patterns that generalize. Capacity is no longer the bottleneck; data is.
Action	Reduce overfitting by getting more training data, adding regularization (weight decay, dropout, data augmentation), or shrinking the model. Stop training earlier if dev loss is rising.

Scenario 3 · Just right (the healthy regime)	
Symptom	Train loss and dev loss both fall steadily during training, with dev loss tracking slightly higher than train loss.
Diagnosis	The model is well-matched to the data. The small gap between train and dev is normal — the model fits training data slightly better simply because it has seen those examples, but it generalizes well to new data.
Action	Stop training when dev loss stops improving. This model is ready for final evaluation on the held-out test set.

Your notebook reads off the diagnosis correctly: *“Training loss and the dev loss come to be similar, case of underfitting, which indicates our network is very tiny, very small, so let’s fix that.”* The fix is in the next chapter.

25. First scale-up: 100 to 300 hidden neurons

Chapter overview

Diagnosed as underfitting, the model needs more capacity. The simplest lever to pull is the width of the hidden layer. This chapter triples it from 100 to 300, re-trains, and re-evaluates. The result is informative even though it is not dramatic: loss does drop, but only slightly. This tells us that the hidden layer was not actually the worst bottleneck. The next chapters identify what the real bottleneck is.

First lever to pull: more hidden units. Triple them.

Cell - Bigger hidden layer

```
C = torch.randn((27, 2), generator=g)
W1 = torch.randn((6, 300), generator=g) # was (6, 100)
b1 = torch.randn(300, generator=g) # 300 biases
W2 = torch.randn((300, 27), generator=g) # 300 inputs into output
b2 = torch.randn(27, generator=g)
parameters = [C, W1, b1, W2, b2]
```

Parameter count jumps to about 10k. Train for 30,000 minibatch steps with LR 0.05 (a bit smaller, since the bigger model is more sensitive). Loss falls, but not by much. We also plot the loss curve:

Cell - Plot the loss over training

```
plt.plot(stepi, lossi)
```

The "hairy band" in the loss curve

Your notebook notes: "The thickness in the bottom signifies the work done on minibatches." Spot on. The loss curve looks like a thick fuzzy line, not a smooth descent. Each step's loss is computed on a different random minibatch, so the loss values bounce up and down by chance. The *trend* is downward, but each individual point is noisy. This is normal minibatch SGD behaviour. If you want a cleaner curve, plot a running average.

Dev loss barely moves. Karpathy diagnoses the new bottleneck: it is not the hidden layer anymore, it is the 2-D embedding. Twenty-seven characters squeezed into a 2-D space cannot be discriminated finely enough for the rest of the network to use.

26. Visualizing the 2-D embedding space

Chapter overview

Before we increase the embedding dimension, the notebook pauses to *look* at what the network learned in 2-D. This is one of the most pedagogically rewarding cells in the whole video. The scatter plot shows that the model spontaneously discovered structure in the data — vowels and consonants form distinct clusters — without anyone telling it what vowels or consonants are. This emergent structure is the whole reason embeddings work.

Before increasing the embedding dimension, the notebook pauses to *look* at what the network learned in 2-D. This is one of the most rewarding cells in the whole video.

Cell - Scatter plot of all 27 character embeddings

```
plt.figure(figsize=(8, 8))
plt.scatter(C[:, 0].data, C[:, 1].data, s=200)
for i in range(C.shape[0]):
    plt.text(C[i, 0].item(), C[i, 1].item(), itos[i],
            ha='center', va='center', color='white')
plt.grid('minor')
```

What you will see in the plot:

- **The vowels cluster together.** a, e, i, o, u end up in roughly one region of the 2-D space. The network discovered, without being told, that vowels play similar grammatical roles in name spelling.
- **The special token .** usually sits off on its own, away from the letters — it has its own special role (start/end), and the network learned it deserves its own region.
- **Other consonant groups** form looser clusters too, often by phonetic similarity.

Why this is the "wow" moment of the lecture

Nobody told the network what vowels are. Nobody encoded any linguistic structure. We just minimized next-character prediction loss with gradient descent, and out fell a representation that puts vowels next to each other. **This is what learned embeddings give you.** Scale this up to GPT and you get embeddings where *king* – *man* + *woman* ~ *queen*. Same machinery, more data.

Why this matters beyond pretty plots

The scatter plot is more than a pedagogical demonstration; it is concrete evidence that the network is learning generalizable structure rather than memorizing specific contexts. Two characters that play similar roles in the language end up with similar embeddings, which means contexts that contain either of them produce similar hidden activations, which means the model's predictions transfer between them. This transfer is precisely the generalization mechanism that count-based models lack, and it is the foundation on which all of modern NLP is built.

But the plot also shows the limitation: 27 distinct things crammed into 2-D inevitably overlap. We can see the network *wants* finer distinctions but does not have the room. Time to give it more room.

27. Second scale-up: 2-D to 10-D embeddings

Chapter overview

The diagnosis of the previous chapter pointed at the embedding dimension as the real bottleneck. This chapter widens the embedding from 2 dimensions to 10, retrains, and observes a dramatic improvement in loss. The chapter also explains why we trim the hidden layer width slightly (from 300 back down to 200) once the embedding is wider, and introduces log-scaling on the loss curve plot.

We give each character a 10-dimensional embedding instead of 2. The input to the hidden layer is now $3 \times 10 = 30$ numbers per example, so W_1 needs to be $(30, \cdot)$. We trim the hidden layer back to 200 since the bigger embedding picked up some of the slack.

Cell - The bigger model

```
C = torch.randn((27, 10), generator=g) # 2 -> 10
W1 = torch.randn((30, 200), generator=g) # input 3*10=30, 200 hidden
b1 = torch.randn(200, generator=g)
W2 = torch.randn((200, 27), generator=g)
b2 = torch.randn(27, generator=g)
parameters = [C, W1, b1, W2, b2]
```

Forward pass changes only in the `.view` call:

```
emb = C[Xtr[ix]] # (32, 3, 10)
h = torch.tanh(emb.view(-1, 30) @ W1 + b1) # 30, not 6
logits = h @ W2 + b2
loss = F.cross_entropy(logits, Ytr[ix])
```


We train for 50,000 minibatch steps at LR 0.01. The loss curve is now plotted on a $\log_{10}$ scale so the late-training plateau is visible:

```
lossi.append(loss.log10().item())
```

Your notebook explains it well: “Let’s plot the log loss instead of normal loss, as log squashes everything in the graph.” Without the log, the early steep drop dominates the plot and you cannot see what is happening at the end.

Why log-scale the loss

Loss curves are notoriously hard to read on a linear scale because the early progress is so much larger than the late progress. The first few hundred steps might take the loss from 17 down to 3, while the next ten thousand steps take it from 3 down to 2.2. On a linear plot the second phase is barely visible. Plotting the log of the loss compresses the early progress and stretches out the late progress, making it easy to see whether you are still making improvements or have hit a plateau.

28. Beating the bigram baseline - first sign of overfitting

Chapter overview

The final chapter of Part IV reports the headline number for the lecture — the MLP beats the bigram baseline of 2.45 with a final dev loss of 2.20 — and observes the first sign that the model is starting to overfit. This is exactly the right moment to pause and reflect: we have a working improvement, but we have also hit the wall of overfitting, which is the natural starting point for Part 3.

Final evaluation on both splits:

Cell - Train and dev loss

```
# Train
emb = C[Xtr]
h = torch.tanh(emb.view(-1, 30) @ W1 + b1)
logits = h @ W2 + b2
loss = F.cross_entropy(logits, Ytr)
loss    # about 2.18

# Dev
emb = C[Xdev]
h = torch.tanh(emb.view(-1, 30) @ W1 + b1)
logits = h @ W2 + b2
loss = F.cross_entropy(logits, Ydev)
loss    # about 2.20
```

Model	Loss
Bigram (counting)	2.45

Bigram (neural)	2.47
MLP, 2-D embedding, 100 hidden	~ 2.30 (underfit)
MLP, 2-D embedding, 300 hidden	~ 2.27 (still bottlenecked by 2-D)
MLP, 10-D embedding, 200 hidden	2.18 train, 2.20 dev

Your notebook records the win and the warning shot: *“we beat the 2.35 mark by having 2.20 and also the train loss and val loss are starting to depart, which means overfitting just started.”*

Reading the train/dev gap

Train 2.18, dev 2.20. The model is now *slightly* better on training data than dev data — the gap is small (0.02) but it is the first time it has appeared. This is the boundary where you stop scaling up blindly and start thinking about regularization and more sophisticated tricks. That is what Part 3 of the series is about: better initialization, batch normalization, and diagnostic plots for activations and gradients.

PART V

Conclusion and reference

Summary of key takeaways, a diagnostic flowchart for future projects, and a glossary of every term used in the lecture.

29. Summary and key takeaways

Chapter overview

This chapter consolidates everything we have built and learned into a single, dense reference. The concepts listed here are not just relevant to this lecture — they will recur in every Karpathy lecture that follows, in every paper you read, and in every model you build. Reread this chapter periodically as you progress through the series. Things that seem obvious now will feel more profound once you have used them in five different contexts.

What we built

A character-level MLP language model following Bengio et al. 2003. Each character is mapped to a learned 10-D embedding, three consecutive embeddings are concatenated into a 30-D context vector, passed through a 200-unit hidden layer with tanh non-linearity, and finally projected to 27 logits. Trained with minibatch SGD on 228k bigrams. Loss: 2.20 on dev, vs 2.45 for the bigram — a real improvement.

The seven concepts that matter most

- **1. Embeddings.** Each token gets a learned dense vector. Indexing a matrix is mathematically the same as multiplying a one-hot vector by it, but vastly more efficient. Same primitive as Word2Vec, GloVe, BERT, GPT — all the way up.
- **2. `.view()` is free.** Changing the shape of a tensor (without changing the data) is metadata-only in PyTorch. Reach for `.view` before `cat` or `unbind`. The `-1` trick handles unknown batch sizes elegantly.
- **3. Always use `F.cross_entropy`.** It is a fused kernel that is faster, more memory-efficient, and numerically stable thanks to the log-sum-exp trick. Never roll your own softmax+log in production.
- **4. Minibatches beat full-batch.** Approximate gradients are cheap. Cheap noisy gradients you can take 1000 of beat one exact gradient you took an hour to compute. Standard size: 32 to 256. The variance in the gradient even acts as a mild regularizer.
- **5. The LR finder.** Do not guess the learning rate. Sweep exponentially from $1e-3$ to 1, plot loss, pick the value in the steep-descent region. Two minutes of work that saves hours of bad runs.
- **6. LR decay.** Once loss plateaus, drop the learning rate by $10\times$ and keep training. The smaller step lets the optimizer settle into a minimum it was overshooting.
- **7. Train / dev / test.** Without these three disjoint splits you literally cannot tell if your model is good or just memorising. Train updates parameters; dev compares models; test evaluates exactly once.

Diagnostic flowchart

Memorize this. It comes up in every project after this one.

- **Train loss high, dev loss high, they are equal → underfit.** Make the model bigger.

- **Train loss low, dev loss much higher** → **overfit**. Regularize, get more data, or shrink the model.
- **Train loss low, dev loss only slightly higher** → **healthy**. Stop.
- **Loss is NaN** → **numerical instability**. Check for missing `F.cross_entropy`, exploding logits, or a learning rate way too high.
- **Loss is not moving** → **learning rate too small or no** `requires_grad`. Sanity check both before anything else.

Where Part 3 picks up

Part 3 (*Building makemore Part 3: Activations & Gradients, BatchNorm*) takes this exact model and asks: why is the initial loss 17 instead of $\log(27) \sim 3.3$? Why does the `tanh` layer saturate so badly at the start? Once you see the activation and gradient distributions, the answer becomes obvious — and the fix (Kaiming init, BatchNorm) is a couple of one-liners that drop the final loss further with no architecture change. Same model, smarter setup.

The single thing to remember if you remember nothing else

Embeddings + hidden layer + softmax + cross-entropy + minibatch SGD *is* the deep-learning loop. Every model from here to a Transformer is a more elaborate forward pass wrapped around exactly this scaffold. If you understand this notebook, the rest of the series is “same loop, fancier forward pass.”

30. Glossary

This glossary defines every technical term used in the handbook in a single, scannable place. If a term ever appears that you do not recognize, look it up here first.

Embedding. A learned dense vector representation of a discrete token (here, a character). The embedding matrix `C` is one of the trainable parameters of the model. Indexing into it (`C[ix]`) is mathematically equivalent to multiplying a one-hot vector by it, but much faster.

Block size / context length. How many previous tokens the model sees when predicting the next one. `block_size = 3` in this notebook.

Hidden layer. An intermediate layer between input and output. In this MLP, it is a single linear layer of size 100 / 200 / 300, followed by a `tanh` non-linearity.

Non-linearity (activation function). A function applied element-wise between linear layers, like `tanh` or `ReLU`. Without one, stacking linear layers is equivalent to a single linear layer.

Logits. The raw, pre-softmax output of a classifier. Real-valued, can be positive or negative. Interpreted as “log-counts.”

Softmax. $\exp(x) / \sum(\exp(x))$. Turns logits into a probability distribution.

Cross-entropy loss. Same thing as average NLL when targets are class indices.

`F.cross_entropy(logits, Y)` in PyTorch fuses softmax, log, and NLL into one numerically stable call.

Log-sum-exp trick. Subtracting the max value from every logit before exponentiating, so the largest exp is $\exp(0) = 1$. Does not change the result mathematically (the constant cancels) but prevents `inf` from overflow. Used by `F.cross_entropy`.

Minibatch. A small random subset of the training data used for one gradient step. Standard sizes are 32 to 256. The full dataset is split into many minibatches; one epoch is one full pass through all of them.

Epoch. One complete pass through the entire training set. In this notebook we do not strictly do epochs — we just sample minibatches with replacement for a fixed number of steps.

`torch.randint`. Generates random integers in a given range. Used to sample minibatch indices.

Learning rate (LR). The size of each gradient step. The most-tuned hyperparameter in deep learning.

LR finder. An empirical technique: sweep through exponentially-spaced learning rates and plot loss vs LR to find the steepest-descent region. Pick the LR in that valley.

LR decay / schedule. Lowering the learning rate during training. A common pattern is “LR=0.1 for most of training, then 0.01 at the end” — used in this notebook.

`torch.linspace`. Generates a 1-D tensor of equally spaced points between two endpoints.

`torch.linspace(-3, 0, 1000)` gives 1000 points from -3 to 0.

`.view()`. Reinterpret a tensor's shape without copying data. Requires the tensor to be contiguous in memory. Use -1 for one dimension to have PyTorch infer it.

`.reshape()`. Like `.view` but falls back to a copy if the tensor is not contiguous. Slightly safer, slightly less explicit.

`torch.cat`. Concatenate a sequence of tensors along a given dimension. All tensors must have the same shape except along that dimension.

`torch.unbind`. Remove a dimension by splitting along it into a tuple of slices. Inverse of `torch.stack`.

Train / dev / test split. Three disjoint subsets of the data: train (for gradient steps), dev/validation (for hyperparameter selection), test (for one-shot final evaluation). 80/10/10 is a common ratio.

Underfitting. Model performance is poor on both train and dev sets, with both losses similar. The model is too small to capture the data.

Overfitting. Train loss is much lower than dev loss. The model has memorized training data but does not generalize.

Parameter count. Sum of `element_size()` over all trainable tensors. For our final model: about 12k. For GPT-3: 175 billion. Same equation, different scale.

Reproducibility (seed). Setting a fixed seed in random-number generators so that re-running the same code produces identical results. Different seeds for different RNGs (PyTorch and Python's `random`) must each be set.

Bengio 2003. *A Neural Probabilistic Language Model* by Bengio, Ducharme, Vincent, Jauvin. The paper that introduced learned word embeddings and the MLP language-modelling blueprint we follow in this notebook.

Advanced indexing. PyTorch's mechanism for using integer tensors as indices into other tensors. `C[X]` where X is a 2-D integer tensor returns a 3-D tensor of looked-up rows.

Stride / storage / view. Internal PyTorch concepts. Storage is the underlying 1-D memory buffer. Stride and shape describe how that buffer is reinterpreted as a multidimensional tensor. `.view` changes shape and stride without touching storage.

Fused kernel. A GPU operation that combines multiple logical steps (e.g. softmax + log + gather) into a single pass over memory, avoiding the intermediate allocations a naive implementation would produce. `F.cross_entropy` is a fused kernel.

Generalization. The ability of a model to perform well on inputs it has never seen during training. Measured by dev or test loss. Generalization is what neural networks have and lookup-table models lack.

Hyperparameter. Any setting of the model or training procedure that is not learned by gradient descent. Examples: hidden size, embedding dim, learning rate, block size, batch size. Tuned on the dev set.

End of Part 2. Part 3 keeps the same network and zooms in on the internals — activation statistics, gradient flow, dead neurons, and BatchNorm. Same model, same loop, sharper microscope.